

Introduction to deep learning- final project report

Authors: Adi Shoham, Lotem Sakira, Gilad Lifshitz.

Abstract .1

In project 1, we address the problem of visual understanding and classification of chessboard images.

Given an image of a chessboard, our system identifies and classifies the individual chess pieces and reconstructs a synthetic representation of the board state.

Rather than training a model from scratch, we leverage and fine-tune a pre-trained deep learning model, thereby reducing training complexity.

Experimental results demonstrate strong performance, with the proposed system achieving an overall accuracy of 95.4%, indicating the effectiveness of our approach.

Introduction .2

Task description .2.1

The task addressed in this project is the automatic visual understanding of chessboard images.

Given a single image of a chessboard, the goal is to accurately identify and classify the individual chess pieces located on the board and to reconstruct a synthetic representation that reflects the same board configuration.

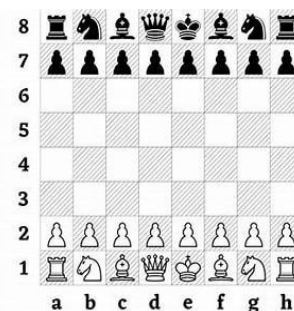
In addition, tiles that could not be recognized had to be classified as out-of-distribution tiles. (for example, if a player's hand hides the tile)

visualization:

Input:



output:



Challenges .2.2

The project presented several challenges that influenced both the design of the system and the choice of methodology:

Limited labeled data in early stages

At the beginning of the project, the amount of labeled data was relatively small, making it impractical to train a deep model from scratch and increasing the risk of overfitting.

Unlabeled OOD samples

In the given dataset, the OOD samples were not labeled. This was challenging because it made OOD recognition much harder.

Severe class imbalance

The dataset contained many empty board squares, while certain chess pieces, such as the black king, appeared very infrequently. This imbalance biased learning toward dominant classes and complicated performance evaluation.

Fine-grained visual distinctions between chess pieces

Many chess pieces share similar shapes and textures, making accurate classification difficult, particularly under challenging lighting conditions or partial occlusions.

Consistency between classification and OOD detection

Designing a system that performs both piece classification and OOD detection required maintaining consistent representations across tasks, rather than treating them as independent problems.

Robustness to real-world variability

Variations in image quality, camera angle, and board appearance introduced noise that the model needed to handle without sacrificing performance.

Main Contributions: **.2.3**

- A labeled dataset of chessboard square images, including manually annotated out-of-distribution (OOD) samples. ○
- A deep learning system for chess piece classification based on pre-trained visual representations. ○
- A DINO-based approach for detecting out-of-distribution squares in chessboard images. ○
- A method for reconstructing a synthetic board representation from model predictions. ○
- An empirical evaluation demonstrating strong performance on both classification and OOD detection tasks. ○

Related work .3

- DINOv2 (ViT-S/14) implementation from Meta AI: ○
<https://github.com/facebookresearch/dinov2>
- Mahalanobis distance-Wikipedia page: [Mahalanobis distance - Wikipedia](#) ○
- Qwen GitHub repository: <https://github.com/QwenLM/Qwen-VL> ○
- Official LoRA implementation (reference): ○
<https://github.com/microsoft/LoRA>

Method .4

Workflow overview: .4.1

Input acquisition ○

A single RGB image of a chessboard is provided as input to the system.

Image validation and preprocessing ○

The input image is validated, converted to RGB format, and prepared for downstream processing.

Board slicing ○

The chessboard image is divided into 64 square tiles (an 8×8 grid), where each tile corresponds to a single board square and is resized to a fixed resolution.

Feature extraction ○

Each tile is passed through a finetuned DINO v2 small backbone, producing a fixed-dimensional embedding that captures semantic visual information.

Chess piece classification ○

The embeddings are used to predict the chess piece class (or an empty square) for each tile using the k-nearest neighbors (knn) method.

Out-of-distribution (OOD) detection ○

In parallel, a dedicated OOD detection dino v2 small net processes each tile and classifies it as either in-distribution or out-of-distribution.

OOD masking ○

Tiles detected as out-of-distribution are explicitly marked and override the corresponding classification predictions.

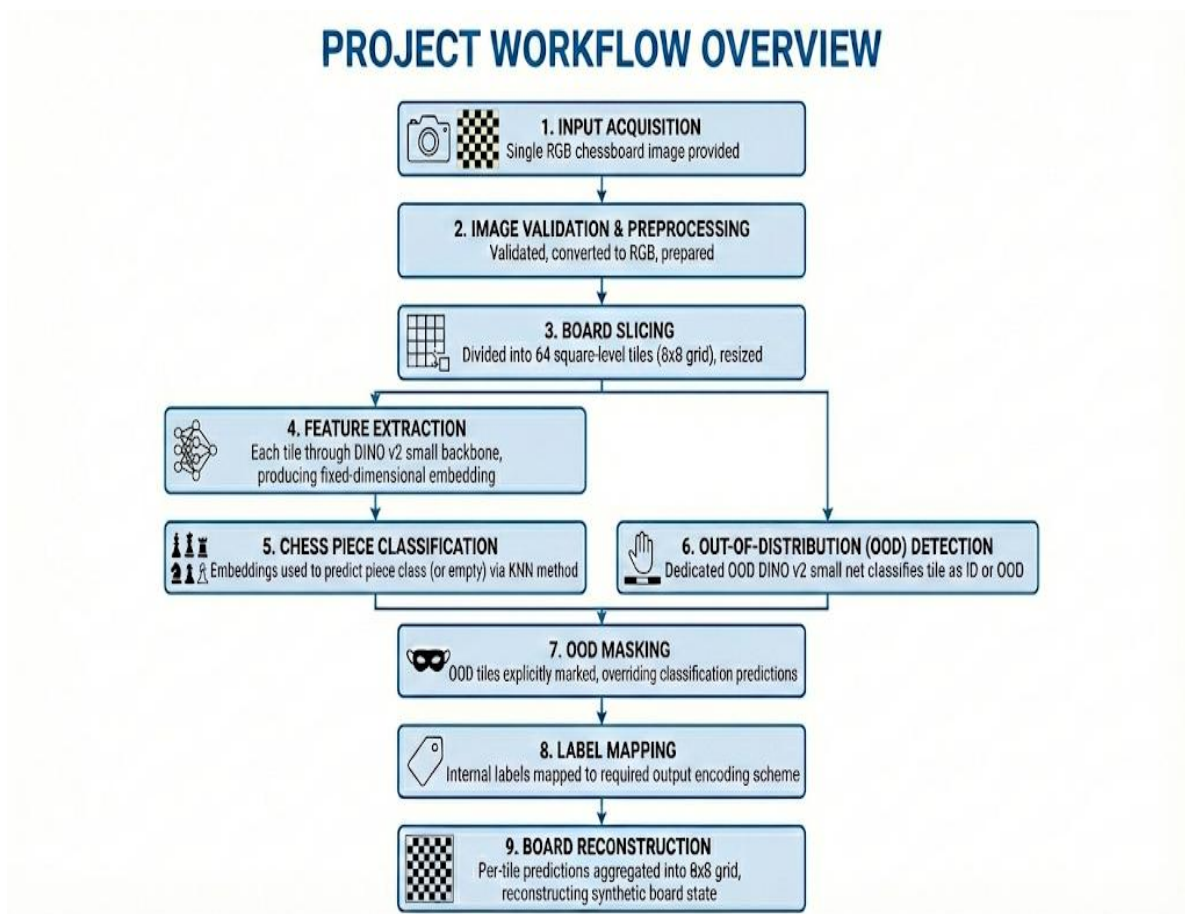
Label mapping ○

Internal prediction labels are mapped to the required output encoding scheme.

Board reconstruction ○

The per-tile predictions are aggregated into an 8×8 grid, reconstructing a synthetic representation of the predicted board state.

Visualization:



Model architecture .4.2

Classification net .4.2.1

Backbone: A fine-tuned DINOv2-small vision transformer (ViT-S/14), pre-trained.

Feature Extraction: For each tile, the DINOv2-small backbone produces a 384-dimensional embedding.

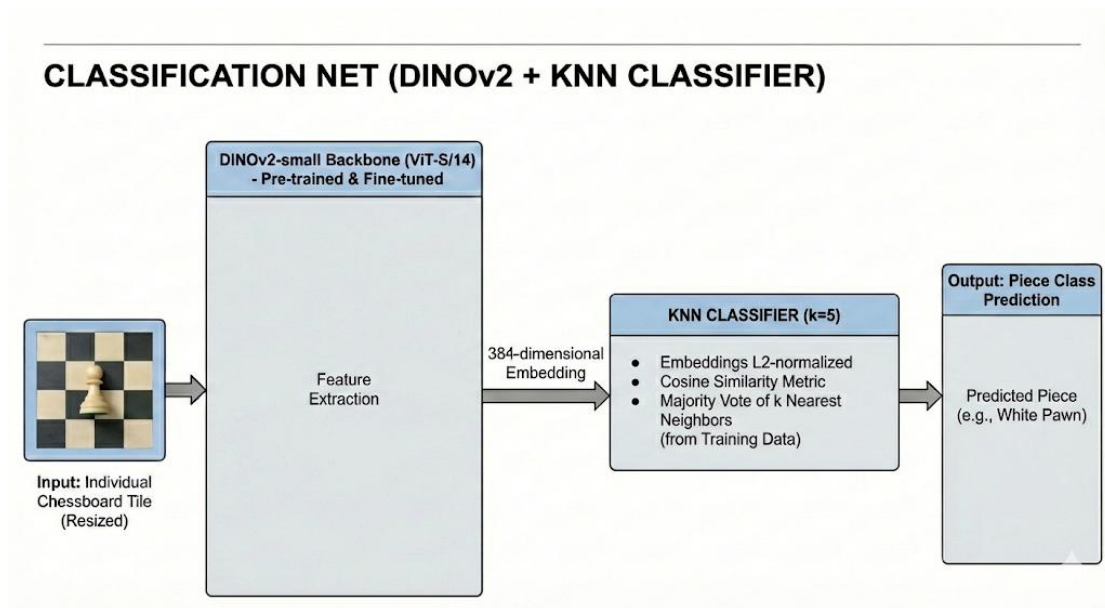
Classifier: A k-nearest neighbors (KNN) classifier operating directly in the embedding space.

No additional trainable classification head is used.

Similarity Metric: Embeddings are L2-normalized, and similarity is computed using cosine similarity.

Prediction Rule: For each tile, the predicted class is the majority label among the k nearest neighbors. (among images from the training data). The value of k is fixed ($k = 5$) across all experiments.

Visualization:



OOD detection: .4.2.2

Backbone: DINOv2-small vision transformer (ViT-S/14).

Input: Individual chessboard tiles, processed one by one.

Feature extraction: Each tile is converted into a 384-dimensional embedding using the DINOv2 backbone.

Classifier head: A small neural network on top of the embeddings:

Linear layer ($384 \rightarrow 192$) ▪

ReLU activation ▪

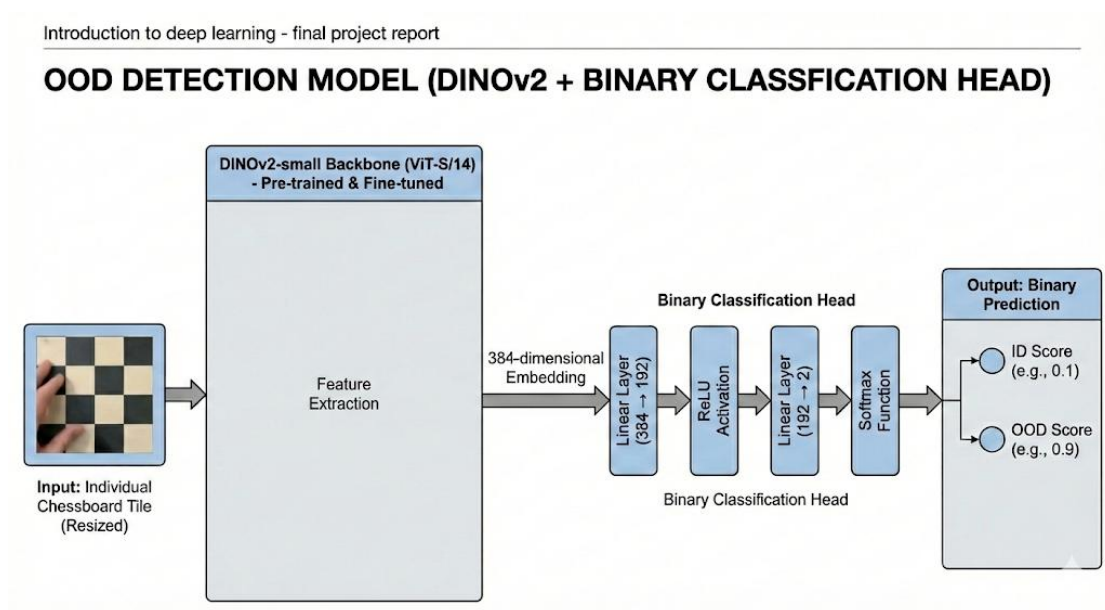
Linear layer ($192 \rightarrow 2$) ▪

Softmax function ▪

Output:

Two scores per tile: ID or OOD. The higher score determines whether the tile is considered out-of-distribution.

Visualization:



Input\output representation .4.3

Input: .4.3.1

- The system's input is a single RGB image of a chessboard.
- The board image is split into 64 square tiles, one for each board position. This is done by cutting the picture into 64 slightly overlapping squares.
- Each tile is resized and normalized before being processed by the model.
- Tiles are handled independently by the system.

Output: .4.3.2

- For each of the 64 tiles, the system outputs a class prediction and a Boolean OOD prediction. If it's OOD, the prediction is set as "OOD". Otherwise, the prediction is set as its class prediction.
- From those 64 class predictions, we construct a FEN string representing the board's state.

With the FEN string, we generate an image of the board's state and output it. ○

Training process .4.4

Classification net training:

We start from a pretrained DINOv2-small vision transformer backbone and train it to produce chess-aware visual embeddings at the tile level. Each tile is treated as a separate input and assigned to one of 13 classes: empty square, six white pieces, or six black pieces.

During training, a lightweight multi-class classification head is attached to the DINOv2 embeddings. The head consists of two fully connected layers with a ReLU activation and dropout ($p = 0.2$) for regularization. This head is used only during training to shape the embedding space and is not used during final inference.

The entire DINOv2 backbone remains trainable, and no layers are frozen. To ensure stable fine-tuning, we use different learning rates for different parts of the model:

- a very small learning rate for the DINOv2 backbone (5×10^{-6}),
- a larger learning rate for the classification head (1×10^{-4}).

This setup allows the pretrained visual features to adapt slightly to the chess domain while enabling fast learning of the task-specific classifier.

The model is trained as a multi-class classification task using cross-entropy loss and optimized with the AdamW optimizer. Training is performed for a small number of epochs, and validation performance is monitored to prevent overfitting.

After training, embeddings are extracted for all tiles in the validation dataset and stored. During inference, chess piece classification is performed using a K-nearest neighbors (KNN) classifier operating directly in the learned embedding space, which proved to be more stable and accurate than using the classification head alone.

OOD detection net training:

We start from a pretrained DINOv2-small backbone and add a small binary classification head.

The entire backbone remains trainable. No layers are frozen during training.

We use different learning rates for different parts of the model:

- a very small learning rate for the DINO backbone (5×10^{-6}).

a larger learning rate for the binary classification head (1×10^{-4}) . ▪

This allows slight adaptation of pretrained features and quick learning of the untrained classification head.

The original multi-class labels are converted to a binary task: Normal chess tiles are treated as in-distribution (ID), and hands and foreign objects are treated as out-of-distribution (OOD). ▪

Dropout (0.3) is applied in the classifier head during training to regularize the OOD decision boundary.

Because OOD samples are rare, we use a class-weighted cross-entropy loss, giving higher weight to the OOD class to reduce class imbalance. Specifically, penalize more for missed OODs. This way, the model is forced to pay attention to the rare ood tiles. ▪

The formula is
$$L = \frac{1}{N} \sum_{i=1}^N \left(-w y_i \log \left(\frac{e^{z_i y_i}}{\sum_{c \in \{0,1\}} e^{z_i c}} \right) \right)$$
 ▪

When N = batch size. ▪

, c = the model's logit (raw score) for sample i and class c . z_i ▪

label: id is 0, and ood is 1. $y_i \in \{0,1\}$ = ▪

= class weight (id=1, ood =8). $w y_i$ ▪

The fraction is the softmax probability of the correct class. ▪

We optimize the model using the AdamW optimizer, updating both the backbone and the classification head during training.

We trained the model for 3 epochs until the validation accuracy began to decrease, to prevent overfitting, as shown in the table below.

Metric	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5
Avg Train Loss	0.0784	0.0471	0.0337	0.0339	0.0278
Val Loss	0.0735	0.0267	0.0215	0.0385	0.0260
Accuracy	0.9782	0.9902	0.9949	0.9852	0.9912

OOD Recall (Caught)	0.9683	0.9524	0.9365	0.9683	0.9524
False Alarms	160	70	34	108	62

Dataset and preprocessing: .5

Dataset description .5.1

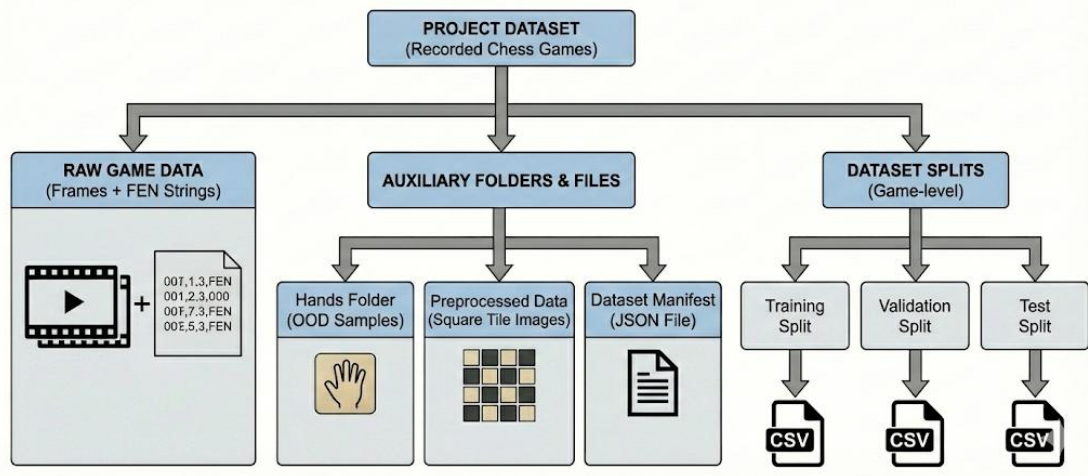
The dataset is constructed from recorded chess games. For each game, we store a sequence of board images (frames) together with their corresponding FEN strings, which describe the board state at each time step.

In addition to the raw game data, the dataset includes several auxiliary folders and files:

- A “hands” folder, containing tile images in which hands or foreign objects appear. This folder is used to mark out-of-distribution (OOD) samples.
- A preprocessed data folder, which contains square tile images extracted from the board frames. Each tile corresponds to a single board position.
- A dataset manifest file (JSON), which describes all tile samples and stores their metadata, including image paths, labels, board identifiers, and OOD flags.
- Three CSV files defining the training, validation, and test splits. These splits are created at the game level to avoid leakage between sets.

Visualization:

DATASET STRUCTURE & ORGANIZATION



Dataset Construction .5.2

First, a hands folder is manually created. This folder contains tile images in which hands or other foreign objects appear and serves as the source for out-of-distribution (OOD) annotations.

Next, as described earlier, each game frame is split into 64 square tiles, one for each board position. Each tile is assigned a piece label according to the corresponding FEN board state.

After tile generation, each tile is checked against the “hands” folder. If the tile’s filename appears in this folder, it is flagged as out-of-distribution. Otherwise, it is treated as in-distribution.

All generated tiles, together with their labels and OOD flags, are stored in the “preprocessed_data” directory. The tiles are then organized into training, validation, and test splits and recorded in the dataset manifest and CSV files.

Dataset augmentation .5.3

To increase data diversity and reduce class imbalance, we apply both manual and automatic augmentation.

For several untaged games with many nearly identical frames, we manually selected representative frames. Frames were chosen at moments where a visible change occurs (for example, when a player moves a piece or a hand appears), and later verified against the game course.

During training, data augmentation is applied to individual tile images:

- Random horizontal and vertical flips. ○
- Small rotations around multiples of 90 degrees with slight jitter. ○
- Color jitter (brightness, contrast, saturation, and hue). ○

Random grayscale conversion with low probability. ○

Experiments .6

Evaluation metrics .6.1

We evaluate the system using several metrics that measure the quality of training, piece classification performance, and out-of-distribution (OOD) detection quality:

Validation loss:

Validation loss is the average cross-entropy loss computed on the validation set. It reflects how well the model's predicted probability distributions match the true labels. This metric is mainly used to monitor convergence and compare training strategies rather than as a final performance measure.

Clean accuracy:

This measures the percentage of board tiles assigned as non-OOD that were correctly classified as the correct piece label. Accuracy is computed at the tile level and reflects how well the model recognizes individual chess pieces.

Balanced accuracy:

Balanced accuracy measures the average recall across all classes. Unlike standard accuracy, it gives equal weight to each class, making it better suited to our setting, where some chess pieces appear much more frequently than others. This metric reflects how well the model performs across all piece types rather than being dominated by common classes.

F1 score:

The F1 score is the harmonic mean of precision and recall. It captures the tradeoff between false positives and false negatives and provides a single score that reflects classification quality. We report a weighted F1 score to account for class imbalance in the dataset.

OOD recall:

OOD recall measures the percentage of true OOD tiles (e.g., hands or foreign objects) correctly detected by the OOD model. A high recall is important to avoid missing hands on the board.

OOD false rejection (false alarm rate):

This metric measures how often regular chess tiles are incorrectly classified as OOD. Lower false rejection indicates that the model does not overreact to clean board tiles.

Overall system accuracy:

This metric evaluates the complete pipeline. For each tile, if the OOD model predicts OOD, the output is set to "OOD". Otherwise, the piece classifier

prediction is used. Overall accuracy is the percentage of tiles for which the final output is correct.

Note: All scores for OOD detections are based on our manual OOD classification and may differ if another person classified the dataset.

Ablation study .6.2

Classification

For the chess piece classification task, we experimented with several strong visual backbones and training strategies, including DINOv2 and Qwen/Qwen3-VL, using both head-only training and backbone fine-tuning.

Rather than performing an extensive architectural ablation, our goal at this stage was pragmatic: identify a classification setup that is accurate, stable, and easy to integrate into the full pipeline. DINOv2-based representations consistently outperformed alternatives, and KNN classification on top of the learned embeddings yielded the most reliable results. Once this configuration was identified, it was fixed and used throughout the rest of the project.

The following table shows a comparison between the different models we tested:

model	strategy	Balanced acc.	F1 score	Val loss
DinoV2 (Backbone)	backbone	95.78%	95.63%	0.1720
DinoV2 (head only)	head only	75.24%	74.09%	0.7692
Qwen3-VL (Head Only)	head only	68.89%	69.46%	0.9517
Qwen3-VL (LoRA)	LoRA	77.82%	78.16%	0.7651

Out-of-Distribution (OOD) Detection

OOD detection required a more exploratory process. Initially, we attempted to solve OOD detection directly on top of the classification embeddings using common confidence-based and distance-based methods. These included

softmax confidence and temperature scaling, KNN vote-based thresholds, cosine similarity thresholds, distance thresholds, and Mahalanobis distance. While some of these methods achieved partial success, their performance was highly sensitive to threshold selection and degraded significantly when hands were present in the training data.

In parallel, we explored a variational autoencoder-based approach, where the model was trained to reconstruct regular chess tiles, and OOD samples were expected to produce higher reconstruction error. This approach failed in practice because the training data still contained hands and foreign objects, allowing the autoencoder to reconstruct OOD samples reasonably well and reducing the separation between ID and OOD tiles.

These failures highlighted a core issue: OOD detection could not be reliably learned when the classification data itself contained OOD samples. As a result, we changed our approach and manually classified the OOD.

After fixing the classification method, we explicitly separated classification and OOD detection into two independent tasks. We cleaned the classification dataset to remove hand tiles and trained a dedicated OOD detection network using a second DINOv2 backbone. This network was trained solely to distinguish between in-distribution chess tiles and OOD samples such as hands and foreign objects.

This separation proved to be the turning point. Once OOD detection was handled by a dedicated network trained on clean, explicitly labeled data, performance became stable and significantly improved across all evaluation metrics. This ablation study demonstrates that enforcing a clear separation between classification and OOD objectives is very important.

This table shows the accuracy of the different configurations we used for classification and OOD detection:

Prediction	OOD_Method	Overall Acc	OOD Recall	False Rejection	Clean Acc (on non-ood guesses)
knn	binary_ood_model	95.41%	99.10%	1.72%	96.97%
knn	softmax	95.02%	40.00%	0.47%	97.09%
mahalanobis	binary_ood_model	94.48%	99.10%	1.72%	96.00%
knn	ensemble	94.29%	7.76%	0.12%	96.95%
mahalanobis	softmax	94.18%	40.00%	0.47%	96.22%
knn	knn	93.84%	10.15%	1.33%	97.59%
mahalanobis	ensemble	93.38%	7.76%	0.12%	96.00%

mahalanobis	knn	93.05%	10.15%	1.33%	96.76%
softmax	binary_ood_model	86.40%	99.10%	1.72%	87.53%
softmax	softmax	86.12%	40.00%	0.47%	87.89%
softmax	ensemble	85.27%	7.76%	0.12%	87.64%
softmax	knn	85.11%	10.15%	1.33%	88.48%
knn	mahalanobis	74.59%	94.03%	25.87%	99.86%
mahalanobis	mahalanobis	74.59%	94.03%	25.87%	99.86%
softmax	mahalanobis	74.30%	94.03%	25.87%	99.46%

An elaboration on our methods explored: .6.3

Throughout the project, we explored multiple design choices for representation learning, classification, and out-of-distribution (OOD) detection. This section summarizes the main approaches we evaluated, emphasizing what worked, what did not, and how these experiments shaped the final system design.

Backbone and training strategies:

We experimented with two main visual backbones: DINOv2 and Qwen/Qwen3-VL-2B-Instruct, using different training strategies.

In one approach, we evaluated using DINOv2 in a head-only configuration. In this setup, the pretrained DINOv2 backbone was kept frozen, and only a lightweight classification head was trained on top of the extracted embeddings. This approach was attractive due to its simplicity and low risk of overfitting. However, the frozen features proved insufficiently specialized for chess tiles. While the model captured general visual patterns, it struggled to distinguish subtle differences between similar pieces and colors, resulting in relatively weak classification performance.

We then fine-tuned the DINOv2 backbone with a small learning rate while training a classification head with a larger learning rate. This allowed the pretrained features to adapt slightly to the chess domain, producing embeddings that were more consistent across board states and more structured for downstream KNN-based classification and OOD detection. This approach consistently outperformed the head-only setup.

Another model we explored was Qwen / Qwen3-VL-2B-Instruct. It is a large multimodal vision-language model designed for general image understanding and reasoning tasks. Unlike DINOv2, which is trained specifically to produce strong visual embeddings through self-supervised learning, Qwen is optimized for broad cross-modal alignment and instruction

following. While this makes it powerful for high-level visual reasoning, it is less specialized for fine-grained visual classification tasks such as distinguishing chess pieces from small board tiles. In our experiments, this difference in design focus led to less consistent embeddings and lower classification performance than DINOv2. We trained its head only, and with LoRA (see following explanation) and dino-v2 small outperformed it in both cases.

A brief explanation of LoRA: LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method used for large pretrained models. Instead of updating all model weights, the original pretrained weights are kept frozen, and small trainable adapter matrices are added to selected layers. These adapters learn a low-rank update to the original weights, allowing the model to adapt to a new task with far fewer trainable parameters. In our setup, LoRA was applied to the Qwen vision model, training only the lightweight adapter parameters and the classification head, while the main backbone remained unchanged.

Classification approaches:

An approach we tried but eventually didn't use is using a classification head: it had 2 layers: the first is half the size of Dino's last, and the second is of size 13. We haven't used it for classification purposes eventually, but it did help us during the training process.

OOD detection experiments:

A significant part of the project focused on OOD detection, and multiple strategies were explored.

Variational Autoencoder-based OOD detection: We experimented with an autoencoder, aiming to reconstruct in-distribution samples well while producing high reconstruction error on tiles containing hands or foreign objects. During inference, tiles with reconstruction error above a threshold were classified as out-of-distribution. In practice, this approach was unstable. Some OOD tiles were reconstructed reasonably well, especially when the hand covered only a small region. At the same time, certain valid chess tiles produced high reconstruction errors due to lighting or uncommon configurations. As a result, reconstruction error alone was not a reliable OOD signal for this task. In addition, we have also tried using the distance of variational encoding to detect outliers directly in the latent space. We experimented with cycle consistency, measuring the 'drift' in the latent representation when an image is repeatedly encoded and decoded, under the assumption that OOD samples would be less stable. Finally, we implemented Likelihood Regret, a metric that quantifies how much the reconstruction loss improves when the latent vector is optimized for a specific image, effectively measuring the model's 'surprise' at the input data. Visualization of these failed attempts can be found at this [drive link](#).

Several KNN-based heuristics were tested:

cosine similarity threshold: If the average cosine similarity of Top-K neighbors $<$ the knn similarity threshold, flag OOD.

distance threshold: If the K-th neighbor's distance $>$ the knn distance threshold, flag OOD.

vote-consensus threshold: If winning class $<$ knn minimal fraction of votes, flag OOD (ambiguous neighborhood).

While these methods worked in some cases, they were highly sensitive to hyperparameter choices and struggled to generalize across different game recordings and lighting conditions.

We also evaluated softmax-based confidence and temperature-scaled confidence from the classification head as OOD indicators. These methods tended to be overconfident on unfamiliar inputs and failed to reliably detect rare OOD cases, such as hands or tools entering the frame.

Additionally, we experimented with Mahalanobis-distance-based classifiers:

As an alternative to KNN-based inference, we evaluated a class-conditional Mahalanobis distance classifier in the DINO embedding space. For each class, a mean embedding vector is estimated from the training data, while a single shared covariance matrix is computed across all classes (tied covariance). At inference time, each tile embedding is assigned to the class whose mean minimizes the Mahalanobis distance, and the distance value is converted into a confidence score. Tiles whose minimum distance exceeds a calibrated threshold are flagged as out-of-distribution (OOD).

This approach assumes that embeddings of each class are approximately Gaussian-distributed in feature space, making Mahalanobis distance a theoretically appealing choice for both classification and OOD detection.

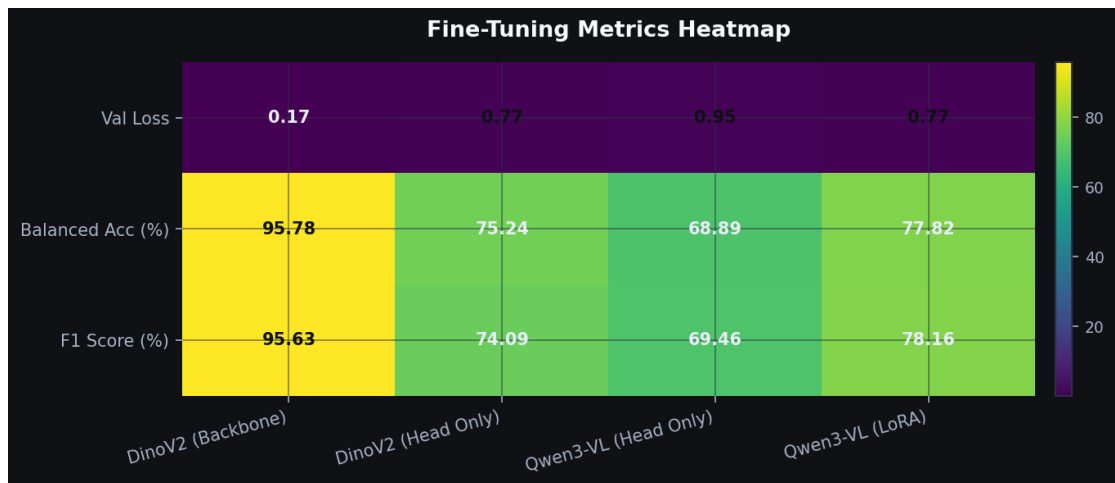
However, in practice, this assumption was only weakly satisfied in our setting. Empirically, Mahalanobis distance underperformed KNN-based methods in both classification accuracy and OOD robustness, and was therefore retained as a secondary inference option rather than the default.

Quantitative results: .6.4

Loss comparison for each model:



Tested models heat map:



Training curves for classification:



t-SNE visualization of the validation data:



Classification and OOD detection methods comparison:



Discussion and limitations .7

Limitations .7.1

A primary limitation of the current pipeline lies in the board preprocessing stage. The board is assumed to occupy the entire image, and tiles are

extracted by uniformly partitioning the image into a 8×8 grid. No explicit corner detection or geometric board localization is performed. As a result, the system is sensitive to misaligned, cropped, or partially visible boards, and performance may degrade when boards do not fully occupy the frame.

A second limitation concerns dataset construction. Additional games and out-of-distribution examples were labeled manually. While effective for rapid iteration, this process does not scale and introduces potential labeling bias.

The reliance on manual curation limits dataset diversity and robustness, particularly for rare or ambiguous failure modes.

Future improvements .7.2

With additional development time, the priority would be incorporating a board localization module, such as corner detection or homography-based alignment, to identify and rectify the chessboard before tile extraction robustly. This would decouple the classifier from strict framing assumptions and significantly improve generalization to unconstrained real-world images.

We tried it with classical line detection but abandoned it to focus on the classifier and OOD detection.

Moreover, currently, tiles are classified independently. Future work could incorporate spatial context by treating the current pipeline of the 8×8 grid of images. A lightweight CNN or GNN could enforce logical consistency and smooth out spurious OOD detections. For example, it is doubtful that a single tile in the middle of the board would be an OOD.

In parallel, the existing data could be leveraged to train an auxiliary model for automatic dataset expansion. Such a model could detect hands and other OOD artifacts, extract informative frames from long game recordings, and assist in labeling new data with minimal human intervention. This semi-automatic pipeline would enable scalable dataset growth, improved coverage of edge cases, and more reliable OOD handling.

Together, these extensions would substantially increase the system's robustness and make it suitable for deployment across diverse real-world chess imagery.

References .8

Website: [Home | CSC-BSR](#)

Git repository: [lotem1995/CSC-BSR](#)